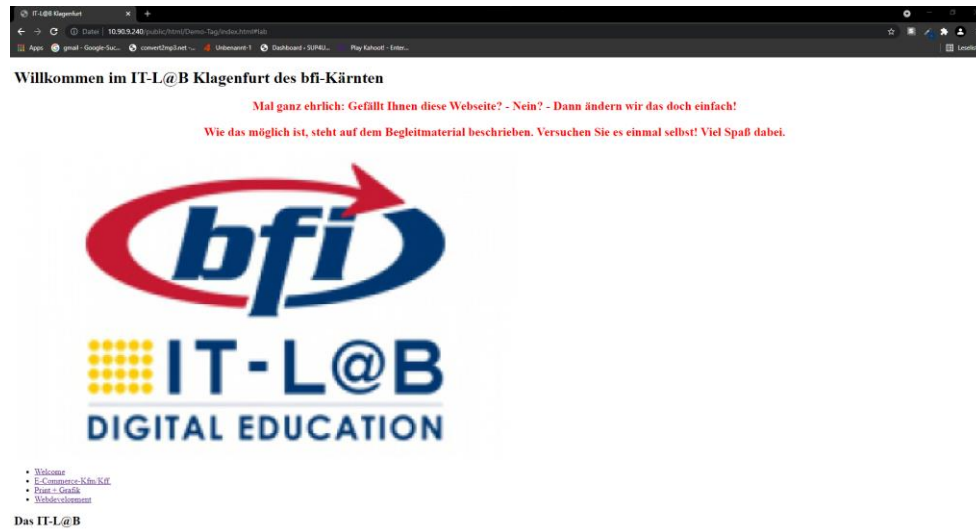


Webdesign bfi-IT-L@B- Einstiegsaufgabe

Ihre Aufgabe: Verschönern Sie die Webseite!



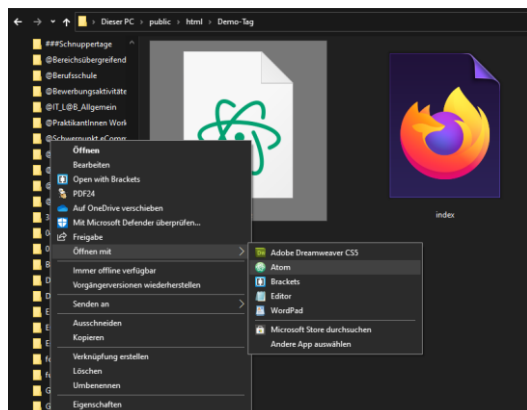
Vor Ihnen liegt eine fertige HTML-Seite, die sich jedoch nicht gerade von ihrer schönsten Seite präsentiert:

- Es gibt kein richtiges Menü.
- Die Bilder sind zu groß.
- Es werden alle Informationen auf einmal angezeigt.
- Und überhaupt: Wo bleibt die Farbe?

Sie brauchen einen Text-Editor!

Öffnen Sie die Datei format.css mit einem geeigneten Editor:

- Microsoft Visual Studio Code
- Brackets
- Notepad++
- Atom



Funktioniert „die Datei“?

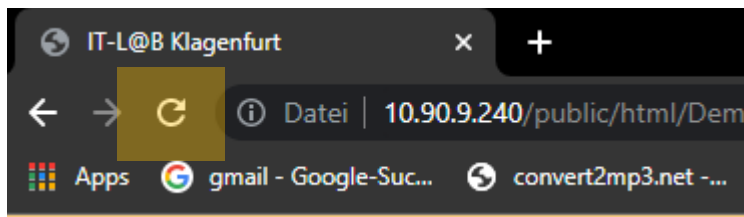
Die Verbindung zu dieser Datei wurde bereits im HTML-Teil angelegt. Achten Sie auf die richtige Schreibweise. Sonst klappt es nicht. Aber probieren wir es aus:

Schreiben Sie folgenden Text in die Datei **formate.css**, um die **Hintergrundfarbe**¹ zu ändern:

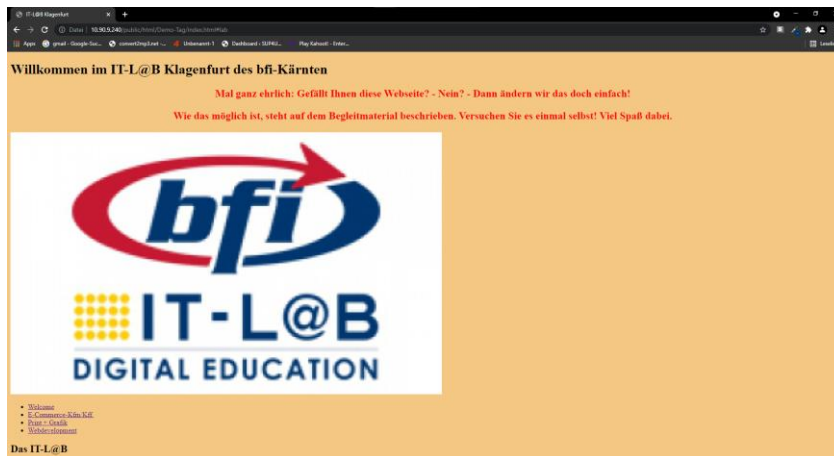
```
body {
    background-color: #F4C882;
}
```

Drücken Sie nun die Tastenkombination **[Strg]+[S]** oder wählen Sie im Menü „File“ (Datei) „save“ (Speichern).

Aktualisieren Sie den Webbrowser mit der Webseite:



Die Seite sollte nun ungefähr so aussehen:



Das Aufmacherbild verkleinern:

Im **HTML-Code** finden wir das Bild in den folgenden Zeilen (bitte NICHT in die CSS-Datei schreiben!):

<header>

...

```

```

</header>

¹ Ein Tipp: <https://html-color-codes.info/> bietet die Möglichkeit, bis zu 16 Mio. Farbcodes zu bestimmen.

Es wird über ein Element mit dem Namen `<img>` eingefügt, welches sich in einem Container mit dem Namen `<header>` befindet.

Für die Formatierung, also die optische Gestaltung einer Webseite verwenden wir die Sprache CSS².

Um unser Bild in CSS anzusprechen, benötigt CSS einen **Selektor**, der die Position der Elemente wiedergibt. Danach werden die CSS-Regeln in einem Block geschweifter Klammern **deklariert**.

```
header img {  
    height: 80px;  
    float: right;  
}
```

Die CSS-Eigenschaft **height** legt die Höhe des Elements fest. Wir verkleinern das Bild auf eine Höhe von 80 Bildschirmpunkten. Die Breite wird automatisch angepasst.

float gibt vor, wie das Element andere Inhalte „umfließt“. Unser Bild erscheint nun auf der rechten Seite.

Wir hätten die Überschrift im Seitenkopf gerne mittig ausgerichtet!

Wir schreiben hinter dem eben geschriebenen Block folgende Zeilen:

```
header {  
    text-align: center;  
}
```

Diese Zeile schreibt vor, dass alle „Inline-Elemente“ – zum Beispiel Text – horizontal zentriert werden. Unsere Überschrift erscheint nun in der Mitte des Fensters.

Kann man auch Schaltflächen im Menü haben?

Ja, das kann man. Wir brauchen jetzt ein paar CSS-Regeln mehr, aber keine Sorge. Das wird klappen!

Zuerst Blickfangpunkte beseitigen! – Für die Darstellung der Listen mit einem Punkt voran ist das Element `<ul>` im HTML-Code verantwortlich. Es sollen aber nur die Listen formatiert werden, die sich in einem Container-Element `<nav>` befinden³. Deswegen muss der CSS-Selektor heißen: **nav ul**

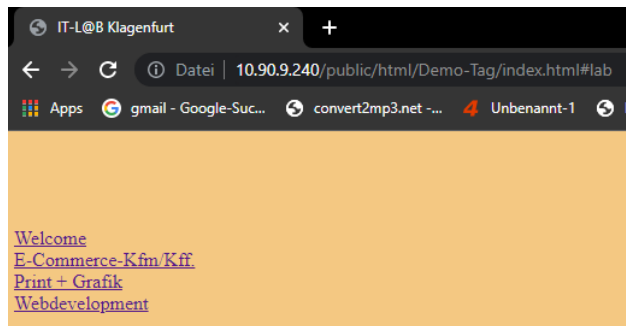
```
nav ul {  
    list-style: none;  
    padding-left: 0px;  
}
```

list-style legt fest, wie der Blickfang der Listenelemente aussehen soll. Die Voreinstellung ist **disc**. Wir wollen aber **keinen** Blickfang. Darum schreiben wir **none**.

Mit dem **padding** – dem Innenabstand – = 0px auf der linken Seite schieben wir die Listenelemente an den linken Seitenrand.

² CSS = Cascaded Style Sheet

³ `<nav>` steht für Navigation (= Menü)



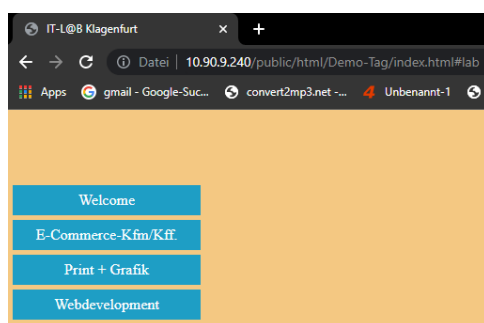
Echte Schaltflächen erzeugen

Eine Schaltfläche kann bunt (Hintergrund, Schrift) sein und hat bestimmte Abmessungen und Abstände zu den anderen „Button“. Dazu kann man Umrahmungen oder Schattierungen ergänzen, die Ecken abrunden und die Flächen animieren.

Wir formatieren das Element `<a>` zur Schaltfläche, denn dann reagiert diese auch beim Klick auf den Rand des Buttons. Natürlich darf das Format nur innerhalb des `<nav>`-Containers wirken. Der Selektor muss also heißen: **`nav a`**.

Schreiben Sie folgenden CSS-Code in die Datei `formate.css`⁴.

```
nav a {
    text-decoration: none;
    display: block;
    height: 25px;
    width: 200px;
    color: white;
    background-color: #1E9EC5;
    padding-top: 7px;
    text-align: center;
    margin-bottom: 5px;
}
```



- **text-decoration** setzt *Über-, Unter- und Durchstreichungen*. Der Wert **`none`** entfernt diese Formate.
- **display: block;** legt fest, dass das Element einen Zeilenumbruch bewirkt (nicht Teil einer Zeile ist). Außerdem kann es in der Breite (**width**) und Höhe (**height**) formatiert werden.
- **color** bestimmt die Schriftfarbe.
- **Background-color** legt die Hintergrundfarbe des Elements fest.
- **padding-top** setzt einen Innenabstand, um auch in vertikaler Richtung den Eindruck der Zentrierung zu erreichen.
- In horizontaler Richtung wird der Text mit **text-align: center;** mittig ausgerichtet.
- Mit einem **margin-bottom: 5px;** legen wir einen Abstand zum nach unten folgenden Element fest.

⁴ Achtung: Immer wieder speichern nicht vergessen und im Webbrowser die Seite zur Prüfung aktualisieren.

Wie die Button auf den Mauszeiger reagieren

Wir brauchen einen eigenen CSS-Regelblock, müssen aber lediglich die CSS-Eigenschaften bearbeiten, die sich bei der Überfahrt mit dem Mauszeiger verändern werden. Alles andere bleibt wie bereits programmiert.

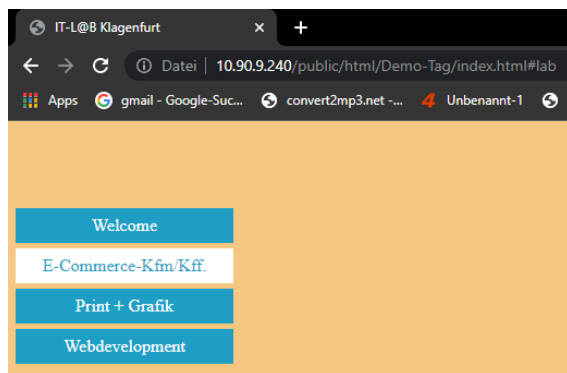
Das Ereignis – Mauszeiger befindet sich über einem Element – wird in CSS mit der dynamischen Pseudoklasse **:hover** erkannt. Diese wird direkt an den Selektor angehängt:

```
nav a: hover {
```

Alles andere erfolgt wie bereits beschrieben. Wir wollen im Beispiel die Schrift- und die Hintergrundfarbe vertauschen:

```
nav a: hover {
    color: #1E9EC5;
    background-color: white;
}
```

Dieser Regelblock wird nur für das Element <a> (innerhalb eines <nav>-Containers) aktiv, über das sich der **Mauszeiger** befindet.



Kann man nur die Inhalte anzeigen, die gewählt wurden?

Ja, natürlich! Auch hier wird wieder eine dynamische Pseudoklasse verwendet. Sie heißt **:target**.

:target wertet den Teil der Adresszeile im Browser aus, der mit einem # beginnt. Unsere Inhalte wurden jeweils in Container-Elementen <article> gespeichert. Wenn wir nur bestimmte Elemente sichtbar machen wollen, müssen wir zuerst alle <article>-Elemente ausblenden:

```
article {
    display: none;
}
```

Ziemlich blöd, nicht wahr? Die Seite ist leer. Darum muss der gewünschte Inhalt wieder sichtbar geschaltet werden. Das erledigt die dynamische Pseudoklasse **:target**.



```

article:target {
    display: block;
}

```



Sie vermissen den roten Text im Seitenkopf?

Wir haben uns erlaubt, diesen ebenfalls mithilfe von CSS auszublenden:

```

header p {
    display: none;
}

```

Experimentieren Sie ein wenig!

Wenn Sie mögen, experimentieren Sie doch ein wenig:

- Verändern Sie die Größe der Abbildungen innerhalb der einzelnen Artikel – Vielleicht können Sie auch hier „Umflüsse“ sinnvoll verwenden?
- Versuchen Sie, Abstände der Textstellen zu den Bildern festzulegen.
- Formatieren Sie andere Textstellen in der Seite.
- Wählen Sie andere Hintergründe
- uvm.

Unsere Berufe brauchen Neugierde, Kreativität und die Freude am Ausprobieren. Ganz wichtig ist jedoch die Motivation!

Kleine Lernzielkontrolle

Erstellen Sie eine Antwortdatei mit dem Namen **[Ihr Name]_Modul1.docx** in Ihrem Praktikanten- bzw. Lehrlingsordner!

Beantworten Sie darin die folgenden Fragen (Achtung: **KEIN** Copy&Paste von anderen Kolleginnen oder Kollegen!!):

Frage 1.1:

Welche CSS-Eigenschaft formatiert die Farbe des Hintergrundes eines Elements?

Frage 1.2:

Ein Farbcode lautet #FFCC00. Auf welche drei Farbkanäle (Grundfarben) wirkt dieser Code? Welche Grundfarbe bekommt welchen (hexadezimalen) Wert zugewiesen?

Frage 1.3:

Beschreiben Sie, was Sie unter einem CSS-Selektor verstehen? Welche Aufgabe hat der Selektor?

Frage 1.4:

Sie haben im Beispiel eine Regel „display: block;“ verwendet. Beschreiben Sie, warum das wichtig war! Was ist der Unterschied zwischen einem Block- und einem Inline-Element (Recherchieren Sie gegebenenfalls in den empfohlenen Internet-Quellen).

Frage 1.5:

Um welche Art von Sprache handelt es sich bei CSS?

Frage 1.6:

Was versteht man bei CSS unter der „Kaskade“?

Frage 1.7:

Wenn man keinen CSS-Code schreibt: Ist eine HTML-Datei dann unformatiert? Wenn nicht: Wie kann man die dann geltenden Formate verändern?

Frage 1.8:

Wozu dient die dynamische Pseudoklasse „:target“? – Beschreiben Sie die Funktion!

Frage 1.9:

Wo (bei welchen Geräten) kann die dynamische Pseudoklasse „:hover“ nicht sinnvoll wirken und warum?

Hilfreiche Quellen für Ihre eigenen Recherchen:

HTML- und CSS-Nachschlagewerke:

- <https://www.w3schools.com/> - Mein Tipp (Tutorials, Nachschlagewerk, Übungsplattform)
- <https://wiki.selfhtml.org/wiki/SELFHTML> - Sehr gute Informationsquelle für CSS und die Kombination von Selektoren
- <https://wiki.selfhtml.org/wiki/CSS/Selektoren>

Farbcodes:

- <https://html-color-codes.info/>
- <https://html-color.codes/>

Bildquellen:

- <https://www.pexels.com/>
- <https://www.publicdomainpictures.net/>
- <https://www.flickr.com/>
- <https://pixabay.com/>

Datenschutzerklärung (Österreich) erstellen:

<https://faresrecht.at/kostenlos-datenschutzerklaerung-erstellen-generator>

Aufgabe für Fortgeschrittene – Eigene kleine Webseite

Überlegen Sie sich ein Thema, zu dem Sie eine Webseite gestalten möchten. Orientieren Sie sich dabei an die im Beispiel verwendete Datei **index.html**. Bei dieser Aufgabe haben Sie freie Wahl des Themas. Die einzige Einschränkung: „Verbotene Inhalte“ (Texte und Bilder etc.) sind unerwünscht.

Wenn Sie die Struktur der Elemente so übernehmen, wie sie ist, sollte sogar die von Ihnen zuvor erstellte CSS-Datei funktionieren.